In [19]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
```

In [20]:
```
df = pd.read_csv("amazon_products_dataset.csv")

print(df.head())
print(df.shape)
```

```
                 product_name     category  actual_price  discount_percent  \
0     Cotton T-Shirt Model 001  Electronics       8068.15              47.0
1  Storage Organizer Model 002         Toys        911.25              27.0
2       Laptop Stand Model 003         Toys       2757.58              50.0
3       Water Bottle Model 004        Books       1142.67               2.0
4         Phone Case Model 005        Books       1037.74               7.0

   discounted_price  review_count  rating
0           4276.12         17938    3.94
1            665.21          2307    3.57
2           1378.79         18998    4.11
3           1119.82          1913    4.06
4            965.10          2435    3.58
(300, 7)
```

In [23]:
```
print("Dataset information:")
df.info()

print("\nMissing values:")
print(df.isnull().sum())

print("\nStatistical summary:")
display(df.describe())

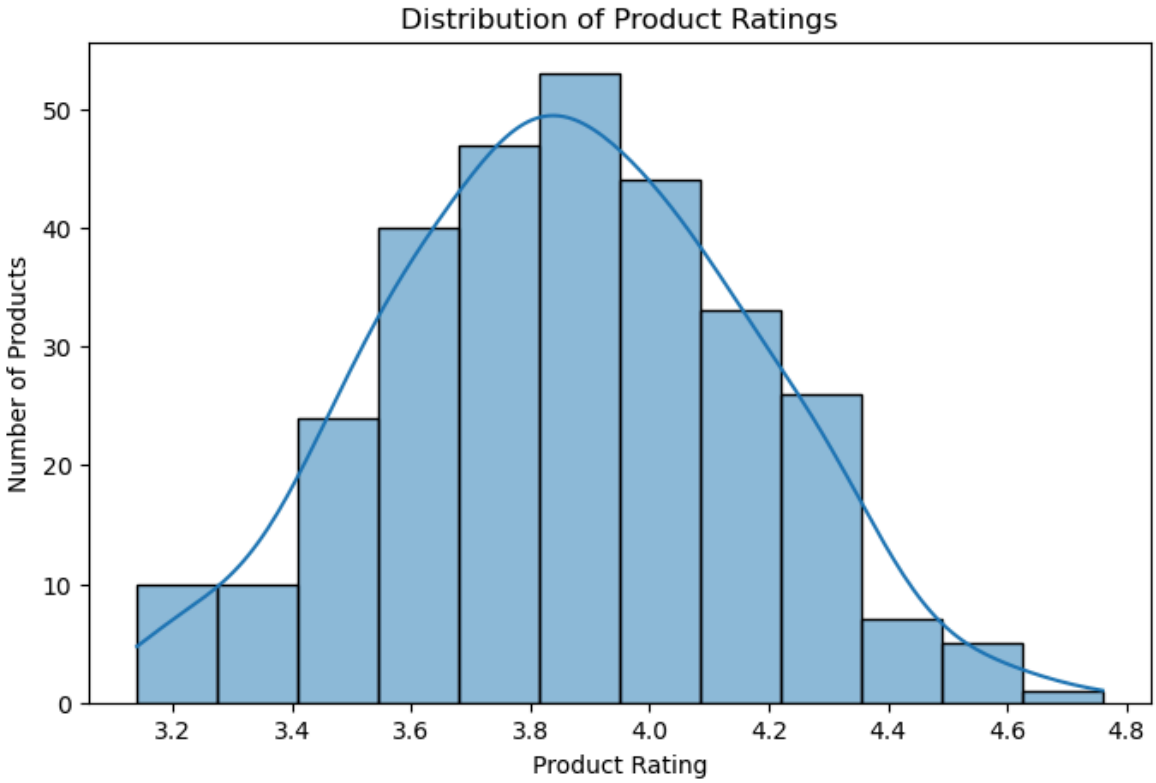
plt.figure(figsize=(8, 5))
sns.histplot(data=df, x="rating", bins=12, kde=True)
plt.title("Distribution of Product Ratings")
plt.xlabel("Product Rating")
plt.ylabel("Number of Products")
plt.show()
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   product_name      300 non-null    object
 1   category          300 non-null    object
 2   actual_price      300 non-null    float64
 3   discount_percent  300 non-null    float64
 4   discounted_price  300 non-null    float64
 5   review_count      300 non-null    int64
 6   rating            300 non-null    float64
dtypes: float64(4), int64(1), object(2)
memory usage: 16.5+ KB

Missing values:
product_name        0
category            0
actual_price        0
discount_percent    0
discounted_price    0
review_count        0
rating              0
dtype: int64

Statistical summary:
```

	actual_price	discount_percent	discounted_price	review_count	rating
count	300.000000	300.000000	300.000000	300.000000	300.000000
mean	2951.455000	28.640000	2121.889733	12417.816667	3.860800
std	3399.910808	17.376715	2603.856762	7179.747755	0.304087
min	120.560000	0.000000	68.150000	114.000000	3.140000
25%	917.120000	14.000000	599.832500	6076.500000	3.650000
50%	1870.100000	27.500000	1286.370000	11924.500000	3.855000
75%	3560.935000	43.250000	2499.360000	18652.500000	4.072500
max	17611.410000	60.000000	15462.260000	24980.000000	4.760000



In [24]:
```
features = [
    "category",
    "actual_price",
    "discount_percent",
    "discounted_price",
    "review_count"
]

X = df[features]
y = df["rating"]

categorical_features = ["category"]

numerical_features = [
    "actual_price",
    "discount_percent",
    "discounted_price",
    "review_count"
]

print("Input features:", features)
print("Target variable: rating")
```

```
Input features: ['category', 'actual_price', 'discount_percent', 'discounted_pric
e', 'review_count']
Target variable: rating
```

In [27]:
```
X = df[["actual_price", "discount_percent", "review_count"]]
y = df["rating"]
```

In [29]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print("Training data:", len(X_train))
print("Testing data:", len(X_test))
```

```
Training data: 240
Testing data: 60
```

In [34]:
```
model = LinearRegression()

model.fit(X_train, y_train)
```

Out[34]:
```
LinearRegression
LinearRegression()
```

In [36]:
```
y_pred = model.predict(X_test)

print("Actual ratings:")
print(y_test.head().values)

print("Predicted ratings:")
print(y_pred[:5].round(2))
```

```
Actual ratings:
[3.79 3.19 3.76 3.87 4.42]
Predicted ratings:
[3.87 3.81 3.97 3.86 3.92]
```

In [38]:
```
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error:", round(mae, 2))
print("R2 Score:", round(r2, 2))
```

```
Mean Absolute Error: 0.24
R2 Score: 0.12
```

In [40]:
```
df["predicted_rating"] = model.predict(X)

top_products = df.sort_values(
    by="predicted_rating", ascending=False
).head(5)

print(top_products[
    ["product_name", "actual_price",
     "discount_percent", "rating", "predicted_rating"]
])
```

```
                         product_name  actual_price  discount_percent  \
11              Coffee Maker Model 012        472.22              58.0
175         Resistance Bands Model 176       1523.40              60.0
94              Notebook Set Model 095       1302.58              58.0
152  Python Programming Book Model 153        506.79              57.0
173      Wireless Headphones Model 174       3207.52              60.0

     rating  predicted_rating
11     3.93          3.975341
175    3.49          3.975284
94     4.00          3.974507
152    3.76          3.973831
173    3.40          3.971421
```

In [42]:
```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Rating")
plt.ylabel("Predicted Rating")
plt.title("Actual vs Predicted Product Ratings")

plt.show()
```

